

The Engineering Platform

A visual feature guide to the self-service developer platform — the **teams-app** portal, the **teams-api**, and the **teams-operator** reconciler — and the ideas behind them.

01 Developer access — kubeconfig & OIDC

03 Creating a project — what the platform does

05 Clarity — the Recent Activity feed

07 Guardrails — policy as admission control

09 Zero trust — identity everywhere

02 The teams-cli — signed, multi-platform

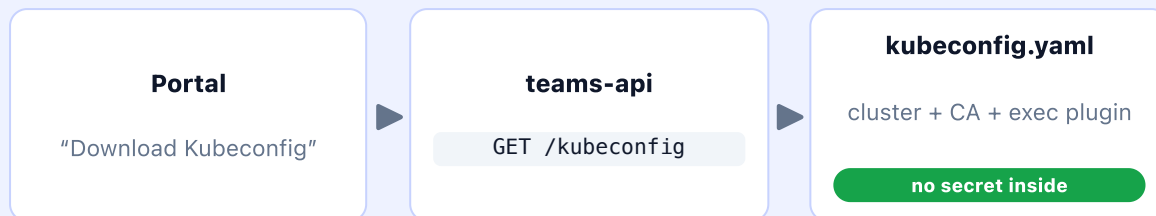
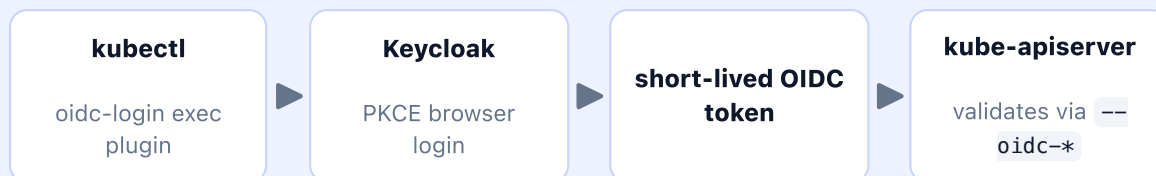
04 State & reconciliation — SQLite → cluster

06 Source-repo integration — GitHub App

08 Not a golden cage — developer freedom

10 Workload secrets — opt-in, injected at admission

One click → a kubeconfig that logs itself in

...then every `kubectl` authenticates on its own

One identity, everywhere

Keycloak (realm `teams`) is the single IdP. The **same login** backs:

`portal (teams-ui)` `kubectl` `Argo CD` `OpenBao UI` `Harbor`
`teams-cli`

Why this matters

- The file carries **no per-user secret** — safe to hand to anyone logged in.
- Tokens are **minted at run time** and short-lived — nothing long-lived to leak.
- Access is **revoked centrally** in Keycloak, instantly.
- Prereq: the `kubectl oidc-login` (kubelogin) plugin.

Pick your flavour

🍏 macOS · Apple Silicon

`teams-cli-darwin-arm64`

🐧 Linux · x86_64

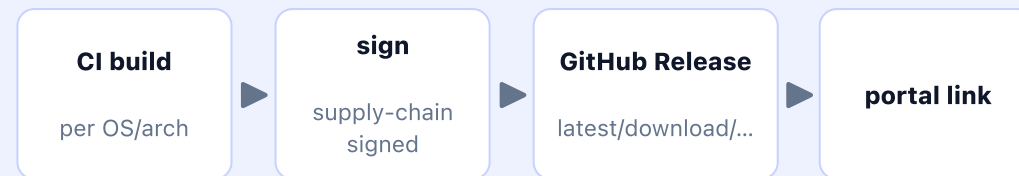
`teams-cli-linux-amd64`

🐧 Linux · arm64

`teams-cli-linux-arm64`

Downloaded straight from the portal header → resolves to the latest signed release asset.

How a flavour is produced & trusted



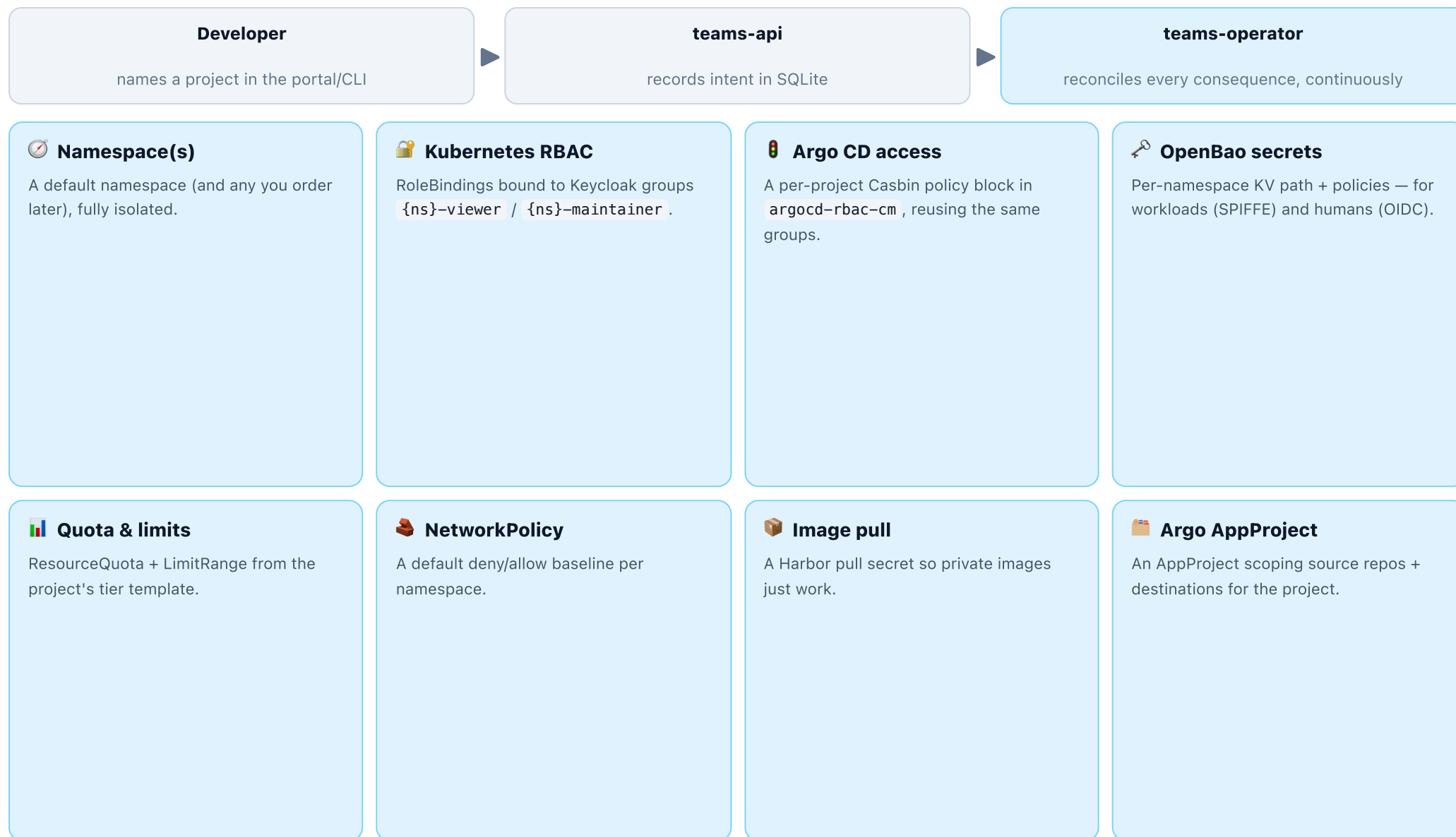
Same platform, second surface

- The CLI talks to the **same teams-api** as the portal — parity, not a subset.
- It logs in with the **same Keycloak OIDC** identity (device/PKCE), so RBAC is identical.
- Great for scripting & CI: create projects, order namespaces, connect repos from a terminal.

portal = clicks

cli = terminal

one API, one identity



The developer expresses **one intent**; the operator translates it into eight coordinated cluster changes — and keeps them true forever after.

TEAMS-API · SQLITE (SOURCE OF INTENT)

projects id · PK name · created_at	project_namespaces namespace · PK ↳ project_id is_default	project_owners ↳ project_id user_id · username
namespace_grants ↳ namespace user_id · role	project_source_repos ↳ project_id repo_url · installation_id connection_id	github_app_connections id · PK ↳ project_id slug · app_id · status
github_app_registrations connection_id code (one-time)	github_connections target · installation_id connection_id (pending)	audit ts · actor · action target · detail

The reconciliation loop (every ~30s)

① **fetch desired** — GET /internal/teams + /internal/access



② **read live** — namespaces, RBAC, quotas, secrets, AppProjects



③ **diff & apply** — create/update only what drifted



④ **emit Events** — per-namespace status & transitions

↺ repeat

The DB never touches the cluster directly. The operator is the **only** writer of cluster state, so intent and reality can never permanently diverge — a hand-edit is reverted next cycle.

Where activity comes from

teams-operator emits Kubernetes **Events** as it provisions (labelled by project)



teams-api aggregates them per project (`/projects/{id}/events`)



teams-app shows **Recent Activity** — open by default, polled live

No black box: provisioning, drift correction and failures surface where the developer already is.

What the developer sees

✓ **Namespace provisioned** project-acme-default · just now

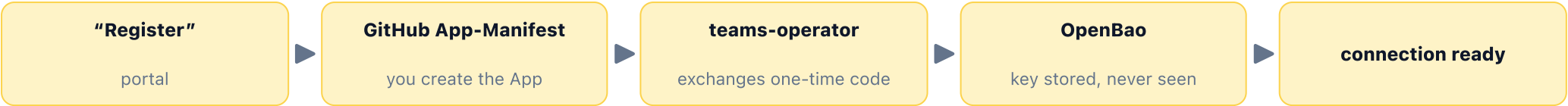
✓ **RBAC synced** viewer/maintainer bound

✓ **OpenBao access ready** secrets path created

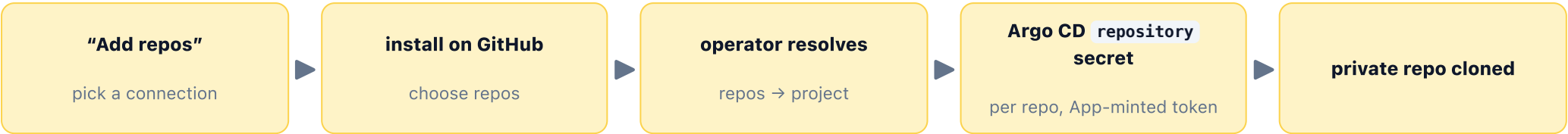
⚠ **Awaiting source repo** register a connection to deploy

A quiet, self-explaining feed — status glyphs + plain language, not raw controller logs.

REGISTER A CONNECTION (ONCE) — NOBODY EVER HANDLES A PRIVATE KEY



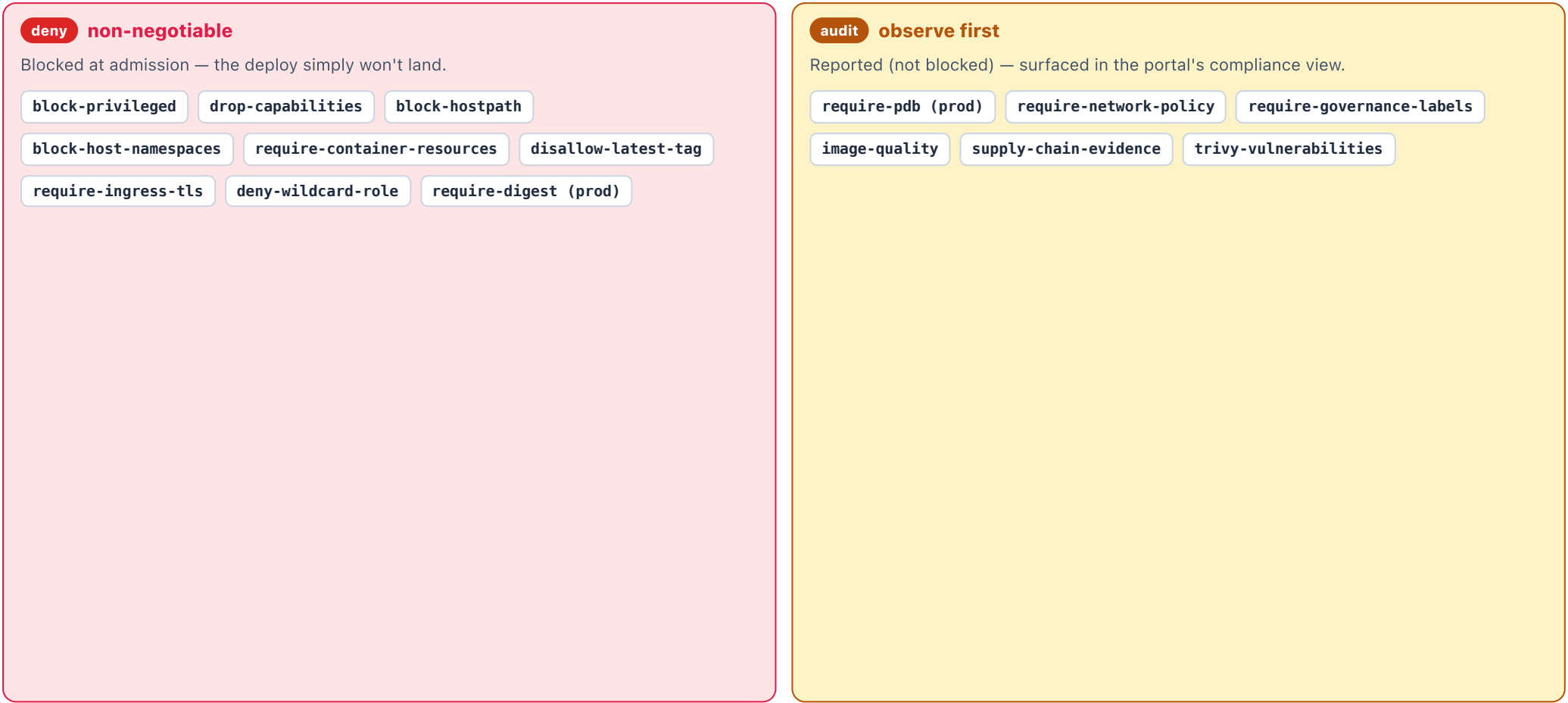
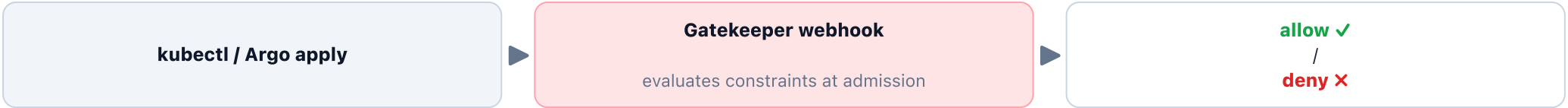
ADD REPOS & DEPLOY



 **teams-api never holds the key** — it only records the one-time manifest code; the operator (which owns the vault write) does the exchange.

 **Ephemeral tokens** — Argo CD mints ~1h GitHub App installation tokens on demand; no standing deploy keys.

 **Bring your own App** — connections are per project, so teams onboard their own GitHub org.



Categories span **security** (privilege, caps, hostpath), **resources** (limits, quotas), **supply chain** (signatures, CVEs, digests) and **networking** (TLS, policies).

Guardrails are **namespace-scoped to tenants** via a label selector — platform components are unaffected; tenants get consistent, automatic enforcement.

 The developer decides

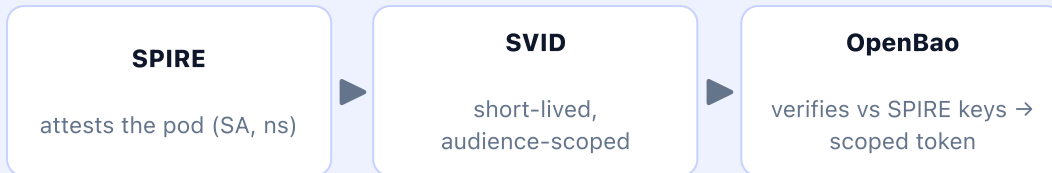
- **Any GitHub repo** — bring your own org via your own GitHub App connection.
- **Your apps, your stack** — the platform ships nothing into your namespace.
- **You own your namespaces** — direct `kubectl` + Argo CD/Rollouts dashboards.
- **Deploy on your schedule** — GitOps from your repo, your commits.
- **Pick your tier** — priority class / quota to fit the workload.
- **Self-service, no tickets** — projects, namespaces, secrets, access on demand.

 The platform guarantees

- **Safe defaults** — RBAC, network policy, quotas provisioned for you.
- **Non-negotiable security** — no privileged pods, no `:latest`, TLS required.
- **Isolation** — one tenant can't reach another's namespace or secrets.
- **Supply-chain hygiene** — signature & CVE checks on images.

Boundaries are few, explicit, and about **safety** — not taste. Everything inside them is the developer's call.

A golden cage optimises for control. This platform optimises for **paved-road autonomy**: the easy path is safe, but the developer is never boxed in.

Workload identity — the no-static-key chain

No password is shared — the workload presents a SPIRE-signed assertion; OpenBao checks the signature, issuer, audience and exact SPIFFE ID. Tenant pods opt in per-workload — sidecars are injected at admission (see next page).

Human identity

Keycloak OIDC is the one front door — portal, kubectl, Argo CD, Harbor, OpenBao UI all federate to it, with short-lived tokens and central revocation.

The principles, made concrete

- **Verify every identity** — SPIFFE for workloads, OIDC for humans.
- **No long-lived credentials** — SVIDs & OIDC tokens expire in minutes/hours.
- **Least privilege** — per-namespace RBAC + per-path OpenBao policies.
- **Assume breach / isolate** — default NetworkPolicy; tenant boundaries enforced.
- **Central trust anchor** — rotate SPIRE/Keycloak keys → access re-derives, no secret sprawl.

🔒 Net effect: **there is no standing secret to steal**. Every access is a freshly-verified, scoped, expiring assertion tied to a proven identity.

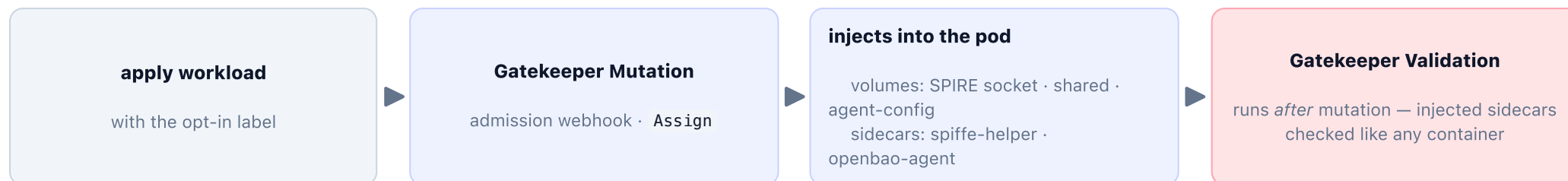
 Opt-in, per workload

Not every workload needs secrets. A developer grants access with **one label** on the workload (Deployment / Rollout):

```
platform.example.com/openbao-access: "true"
```

 No label → nothing injected

Without the label a pod gets **no** sidecars, **no** volumes and **no** OpenBao path — least privilege by default. Secret access is the developer's explicit, deliberate choice.

AT ADMISSION — GATEKEEPER MUTATES THE POD, WITH ZERO APP-CONTAINER CHANGES**AT RUNTIME — THE SIDECARS DO THE WORK; THE APP JUST CALLS LOCALHOST**